CSC 246 Midterm Review

Boyu Zhang

1 Duality and Kernel

Consider the update rule of a perceptron algorithm f

if $y^n f(\boldsymbol{x}^n) = y^n \boldsymbol{w}^T \cdot \boldsymbol{x}^n < 0$ **then**
  $\boldsymbol{w} = \boldsymbol{w} + y^n \boldsymbol{x}^n$
end if

1.1

Give a new update rule of f in its dual form by writing $\boldsymbol{w}$ as a weighted sum of training items.

Answer:

$$\boldsymbol{w} = \sum_n \alpha^n y^n \boldsymbol{x}^n \tag{1}$$

where α^n shows how often was $\boldsymbol{x}^n$ misclassified.

if $y^m \sum_n \alpha^n y^n \boldsymbol{x}^n \cdot \boldsymbol{x}^m < 0$ **then**
  $a^m += 1$
end if

The dual form will be advantageous when the number of input features (length of input vectors) [ ] the number of training examples we have (fill in the blank with >>, <<, or =).

Answer: >>. When the number of input features >> the number of training examples, we will have less α to learn in the dual form. If using the normal perceptron, there will be more parameters in $\boldsymbol{w}$.

1.2

Briefly explain why dual perceptron can be extended to kernel perceptron.

Answer: Given a kernel $\boldsymbol{K}$ where $\boldsymbol{K}_{ij} = \phi(\boldsymbol{x}_i) \cdot \phi(\boldsymbol{x}_j)$, it defines the value of the dot product of the input vectors after the transformation ϕ (but not the transformed vectors $\phi(\boldsymbol{x})$ themselves). In the dual form, we only need to calculate the inner products during training, and it makes kernel methods applicable.

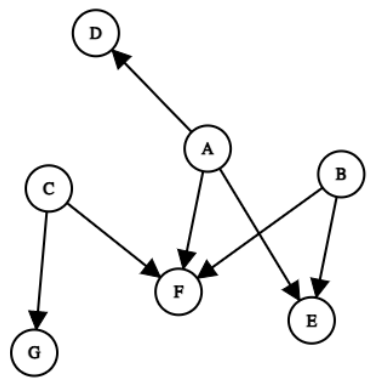

Figure 1: The Belief Net for Question 2.

2 Graphical Models

Consider the belief network in Figure 1.

2.1

If node C is observed, which nodes are dependent
on A: D, E, and F.
on B: E and F.
on D: A, E, and F.
on E: A, B, D, and F.
on F: A, B, D, and E.

If node C and D is observed, which nodes are dependent
on A: E and F.
on B: E and F.
on D: A, E, and F.
on E: A, B, and F.
on F: A, B, and E.

2.2

Convert the belief network in Figure 1 to a factor graph. **Answer**: see Figure 2.

2.3

Convert the belief network in Figure 1 to a dependency graph. **Answer**: see Figure 3.

2.4

Derive a tree decomposition of the dependency graph in Figure 3. **Answer**: see Figure 4.

2.5

Derive an acyclic factor graph based on Figure 4. Give equations for the factors in terms of the probabilities of the original Bayesian network in Figure 1. **Answer**: see Figure 5. $f(AD) = P(A) \times P(D \mid A)$, $f(ABE) = P(B) \times P(E \mid AB)$, $f(ABCF) = P(C) \times P(F \mid ABC)$, and $f(CG) = P(G \mid C)$.

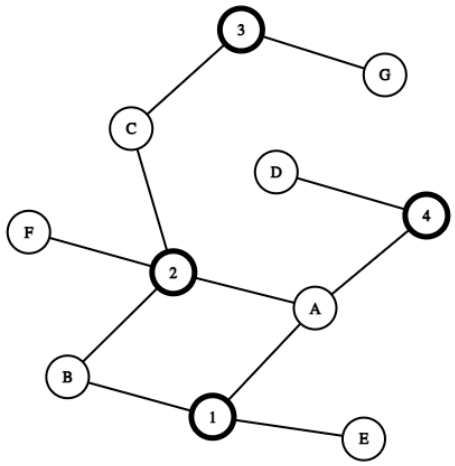

Figure 2: The factor graph equivalent for Figure 1. Bold nodes 1 to 4 are factors.

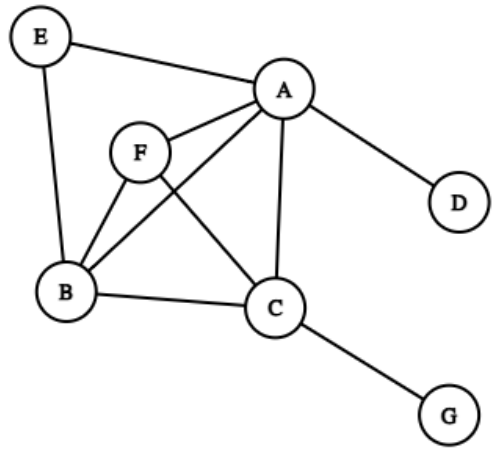

Figure 3: The dependency graph equivalent for Figure 1.

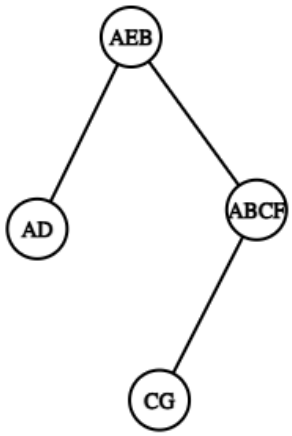

Figure 4: The tree decomposition of the dependency graph in Figure 3.

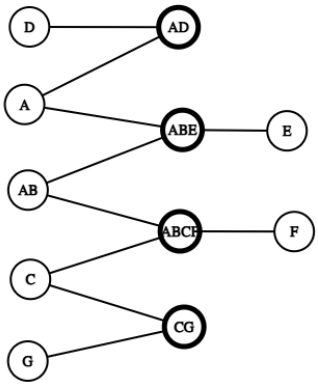

Figure 5: The acyclic factor graph from Figure 4. Bold nodes are factors.

3 Backprop

3.1

Consider a fully-connected feed-forward neural network. Let z_j^l denote the weighted sum at an arbitrary node j at layer l, where $z_j^t = \sum_i w_{ji}^l a_i^{l-1}$. Let a_i^{l-1} denote the output (after activation) of node i from the previous layer. Define z_k^{l+1} and w_{kj}^{l+1} similarly, indexing layer $l+1$ with k. Given a loss L, $\frac{\partial L}{\partial z_k^{l+1}}$, and the derivative of the activation function, g', derive $\frac{\partial L}{\partial w_{ji}^l}$. Show all necessary steps.

Answer:

$$\frac{\partial L}{\partial w_{ji}^l} = \frac{\partial L}{\partial z_j^l}\frac{\partial z_j^l}{\partial w_{ji}^l} = \frac{\partial L}{\partial z_j^l} a_i^{l-1} \tag{2}$$

$$\frac{\partial L}{\partial z_j^l} = \frac{\partial L}{\partial a_j^l}\frac{\partial a_j^l}{\partial z_j^l} = \frac{\partial L}{\partial a_j^l} g'(z_j^l) \tag{3}$$

$$\frac{\partial L}{\partial a_j^l} = \sum_k \frac{\partial L}{\partial z_k^{l+1}}\frac{\partial z_k^{l+1}}{\partial a_j^l} = \sum_k \frac{\partial L}{\partial z_k^{l+1}} w_{kj}^{l+1} \tag{4}$$

Thus,

$$\frac{\partial L}{\partial w_{ji}^l} = a_i^{l-1} g'(z_j^l) \sum_k \frac{\partial L}{\partial z_k^{l+1}} w_{kj}^{l+1} \tag{5}$$

3.2

Which term(s) in your answer should be saved temporarily as cache, be used in the gradient calculation for the layer $l-1$, and facilitate the dynamic programming nature of backprop?

Answer:

$$\frac{\partial L}{\partial z_j^l} = g'(z_j^l) \sum_k \frac{\partial L}{\partial z_k^{l+1}} w_{kj}^{l+1} \tag{6}$$

3.3

Give the vectorized form of your answer, $\frac{\partial L}{\partial \boldsymbol{W}^l}$. Notice that w_{ji}^l refers to the entry at the jth row and ith column of $\boldsymbol{W}^l$. Do NOT use any computation trick such as broadcasting.

Answer:

$$[\boldsymbol{W}^{l+1}]^T \cdot \frac{\partial L}{\partial \boldsymbol{z}^{l+1}} \odot \boldsymbol{g}'(\boldsymbol{z}^l) \cdot \boldsymbol{a}^{l-1} \tag{7}$$

$$[k \times j]^T \cdot [k] \odot [j] \cdot [i] \tag{8}$$

where $\odot$ denotes element-wise products.

4 Binary Classification

Consider a naive Bayesian classifier with the following objective:

$$y \leftarrow \arg\max_y \frac{P(y)\prod_i P(x_i \mid y)}{\sum_y P(y)\prod_i P(x_i \mid y)} \tag{9}$$

with the conditions

1. In the case of binary classification, $y = \{0, 1\}$. y is governed by a Bernoulli distribution with the parameter $\pi = P(y = 1)$.
2. $\boldsymbol{x} = [x_0, x_1, ..., x_n]$ a continuous random variable. For all i and j s.t. $i \neq j$, x_i and x_j are conditionally independent given y.
3. For each x_i, $P(x_i \mid y = y_k)$ is a Gaussian distribution of the form $N(\mu_{ik}, \sigma_i)$.

Show that this naive Bayesian classifier is the same as logistic regression.

Answer:

$$\begin{aligned} P(y = 1 \mid x) &= \frac{P(y = 1)P(\boldsymbol{x} \mid y = 1)}{P(y = 1)P(\boldsymbol{x} \mid y = 1) + P(y = 0)P(\boldsymbol{x} \mid y = 0)} \\ &= \frac{1}{1 + \exp\left(\ln \frac{P(y=0)P(\boldsymbol{x}|y=0)}{P(y=1)P(\boldsymbol{x}|y=1)}\right)} \\ &= \frac{1}{1 + \exp\left(\ln \frac{1-\pi}{\pi} + \sum_i \ln \frac{P(x_i|y=0)}{P(x_i|y=1)}\right)} \end{aligned} \tag{10}$$

$$\begin{aligned} \sum_i \ln \frac{P(x_i \mid y = 0)}{P(x_i \mid y = 1)} &= \sum_i \ln \frac{\frac{1}{\sqrt{2\pi\sigma_i^2}} \exp\left(\frac{-(x_i-\mu_{i0})^2}{2\sigma_i^2}\right)}{\frac{1}{\sqrt{2\pi\sigma_i^2}} \exp\left(\frac{-(x_i-\mu_{i1})^2}{2\sigma_i^2}\right)} \\ &= \sum_i \ln \exp\left(\frac{(x_i - \mu_{i1})^2 - (x_i - \mu_{i0})^2}{2\sigma_i^2}\right) \\ &= \sum_i \left(\frac{(x_i - \mu_{i1})^2 - (x_i - \mu_{i0})^2}{2\sigma_i^2}\right) \\ &= \sum_i \left(\frac{2x_i(\mu_{i0} - \mu_{i1}) + \mu_{i1}^2 - \mu_{i0}^2}{2\sigma_i^2}\right) \\ &= \sum_i \left(\frac{x_i(\mu_{i0} - \mu_{i1})}{\sigma_i^2} + \frac{\mu_{i1}^2 - \mu_{i0}^2}{2\sigma_i^2}\right) \end{aligned} \tag{11}$$

Notice that 11 is a linear weighted sum of x_i, and by substituting it back to 10, we have

$$\begin{aligned} P(y = 1 \mid x) &= \frac{1}{1 + \exp\left(\ln \frac{1-\pi}{\pi} + \sum_i \left(\frac{x_i(\mu_{i0}-\mu_{i1})}{\sigma_i^2} + \frac{\mu_{i1}^2-\mu_{i0}^2}{2\sigma_i^2}\right)\right)} \\ &= \frac{1}{1 + \exp\left(w_0 + \sum_i w_i x_i\right)} \end{aligned} \tag{12}$$

This is exactly the form of logistic regressions.

5 HMM

We observed a sequence $[x_1, ..., x_i, ..., x_n]$. Given a set of discrete hidden states values $\{z^1, ..., z^k, ..., z^m\}$, transition probabilities $t(z_i \mid z_{i-1})$, and emission probabilities $e(x_i \mid z_i)$.

5.1 Evaluating

Find the probability of the observed sequence, $P([x_i]_{i=1}^n)$. Describe a naive algorithm for this problem by summing over all possible hidden state values at all steps and analyze its complexity.

Answer: $O(m^n)$

$$P([x_i]_{i=1}^n) = \sum_{z_i} P([x_i]_{i=1}^n, [z_i]_{i=1}^n) = \sum_{z_i} P(z_0) \prod_1^n P(z_i \mid z_{i-1}) \prod_0^n e(x_i \mid z_i) \tag{13}$$

Give an efficient dynamic programming algorithm (pseudocode) for this problem and analyze its complexity.

Answer:

```
initialize α[n+1][m]
α[0][:] ← P(z_0)
for i = 1 to n do
  for z = z^1 to z^m do
    for z' = z^1 to z^m do
      α[i][z] += α[i−1][z'] × t(z | z') × e(x[i] | z)
    end for
  end for
end for
return ∑_{z_n} α[n][z_n]
```

5.2 Decoding

Find *the most likely* hidden state sequence given the observed sequence. Describe a naive algorithm for this problem by calculating all the possible permutations of the hidden states and analyze its complexity.

Answer: $O(m^n)$

Give an efficient dynamic programming algorithm (pseudocode) for this problem and analyze its complexity.

Answer:

```
initialize δ[n+1][m]
δ[0][:] ← P(z_0)
for i = 1 to n do
  for z = z^1 to z^m do
    for z' = z^1 to z^m do
      δ[i][z]max = δ[i−1][z'] × t(z | z') × e(x[i] | z)
    end for
  end for
end for
return arg max δ[i][:] for i = 1 to n
```